MCA LAB

Timer, String, Memory and I2C Programs – Code Snippets and Expected Outputs

List of Programs:

1. Generate a square wave of 2 ms using timers and port pins.
    a. Square wave of 2 ms using a timer and a port pin.
    b. Receive the string "MANIPAL"; if the character 'L' exists, generate a square wave of 4 ms.
2. Receive two strings and compare them. If they are the same, blink the LED of the port; if not, blink the LED on P2.2.
3. Store 'N' numbers from a memory location (say 0x4000 0000). Find the sum of odd numbers and even numbers and display it on the console.
4. Display the string transmitted using I2C from the controller on the console / receiver.

Board: LPC2148 (ARM7TDMI-S)
Software: Keil µVision 4 (outputs observed in the µVision debugger)

Program 1: Square Wave Generation Using Timers

Problem Statement

Generate a square wave of 2 ms using timers and port pins. Receive the string "MANIPAL"; if the character 'L' exists, generate a square wave of 4 ms.

1a. Square Wave of 2 ms

Code Snippets:

```c
#include <lpc214x.h>

void delay_1ms_015(void){
  T0TCR = 0x02;           // reset timer
  T0PR = 0;               // no prescaler
  T0MR0 = 15000 - 1;      // 1 ms at PCLK 15 MHz
  T0MCR = 0x04;           // stop on match
  T0TCR = 0x01;           // start timer
  while(T0TCR & 0x01);    // wait till done
}

int main(void){
  VPBDIV = 0x00;           // PCLK = 15 MHz
  IODIR0 |= (1<<10);       // P0.10 output

  while(1){
    IOSET0 = (1<<10);    // HIGH
    delay_1ms_015();
    IOCLR0 = (1<<10);    // LOW
    delay_1ms_015();
  }
}
```

Fig: 2 ms Square Wave C code snippet

Output Screenshot

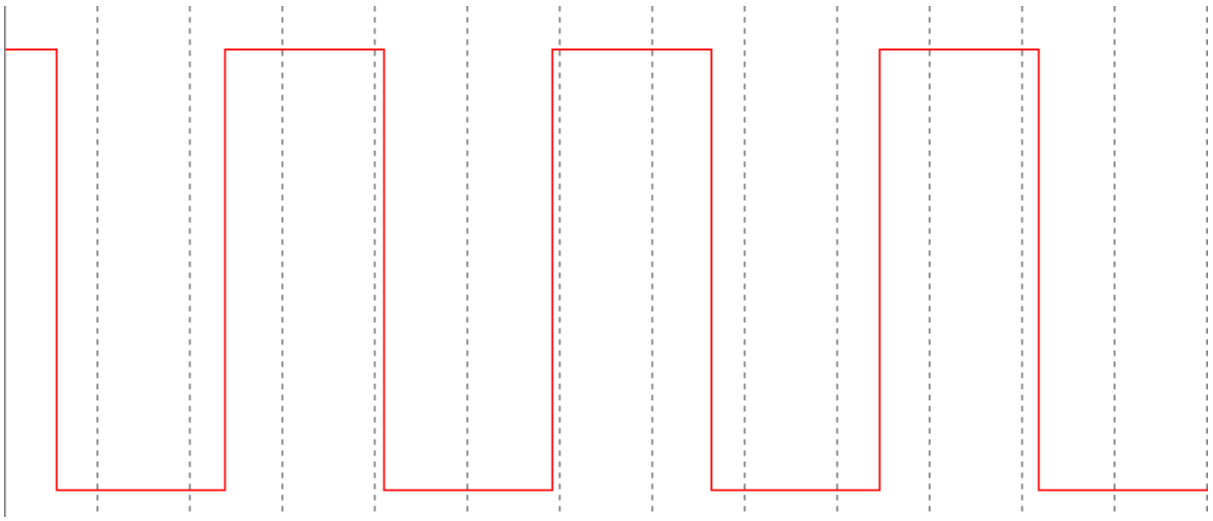


Fig: 2 ms Square Wave C

Expected Output:

Timer0 is configured to give a 1 ms delay (with PCLK = 15 MHz, T0MR0 = 15000 and PR = 0) and the port pin (e.g. P0.10) is toggled on every match. This gives a square wave with a period of 2 ms (1 ms HIGH and 1 ms LOW), i.e. a frequency of 500 Hz and 50% duty cycle.

In Keil µVision 4 debug mode, the pin is added to the Logic Analyzer (View → Analysis Windows → Logic Analyzer). The waveform shows a square wave, and the time between two rising edges measured with the cursors is 2 ms. The pin also toggles in the Peripherals → GPIO → Port 0 window.

Parameter	Value	Remark
Period	2 ms	1 ms HIGH + 1 ms LOW
Frequency	500 Hz	1 / 2 ms
Duty cycle	50%	Equal ON and OFF time
Timer0 match (T0MR0)	15000	For 1 ms at PCLK = 15 MHz

1b. Square Wave of 4 ms if Character 'L' Exists

Code Snippets:

Lcharecter.c* Startup.s

```c
#include <lpc214x.h>

char str[20];

void delay_1ms_015(void){
  T0TCR = 0x02;          // reset timer
  T0PR = 0;
  T0MR0 = 15000 - 1;     // 1 ms
  T0MCR = 0x04;          // stop on match
  T0TCR = 0x01;          // start
  while(T0TCR & 0x01);
}
void uart_tx_015(char *s){
  while(*s){
    while(!(U0LSR & 0x20));
    U0THR = *s++;
  }
}
int main(void){
  int i = 0, ms = 1;
  VPBDIV = 0x00;          // PCLK = 15 MHz
  IODIR0 |= (1<<10);      // P0.10 output
  PINSEL0 |= 0x05;        // P0.0 TXD0, P0.1 RXD0
  U0LCR = 0x83;           // 8N1, DLAB = 1
  U0DLL = 97;             // 9600 baud
  U0LCR = 0x03;

  uart_tx_015("Enter string : ");
  while(1){
    while(!(U0LSR & 0x01));
    str[i] = U0RBR;
    if(str[i] == '\r') break;   // Enter pressed
    U0THR = str[i];             // echo
    i++;
  }
  str[i] = '\0';

  for(i = 0; str[i] != '\0'; i++){
    if(str[i] == 'L'){
      ms = 2;                 // 2 ms half -> 4 ms wave
    }
  }

  if(ms == 2) uart_tx_015("\r\n'L' found -> 4 ms square wave\r\n");
  else        uart_tx_015("\r\n'L' not found -> 2 ms square wave\r\n");

  while(1){
    IOSET0 = (1<<10);
    for(i = 0; i < ms; i++) delay_1ms_015();
    IOCLR0 = (1<<10);
    for(i = 0; i < ms; i++) delay_1ms_015();
  }
}
```

Fig: String Check Square Wave C code snippet

Output Screenshot:

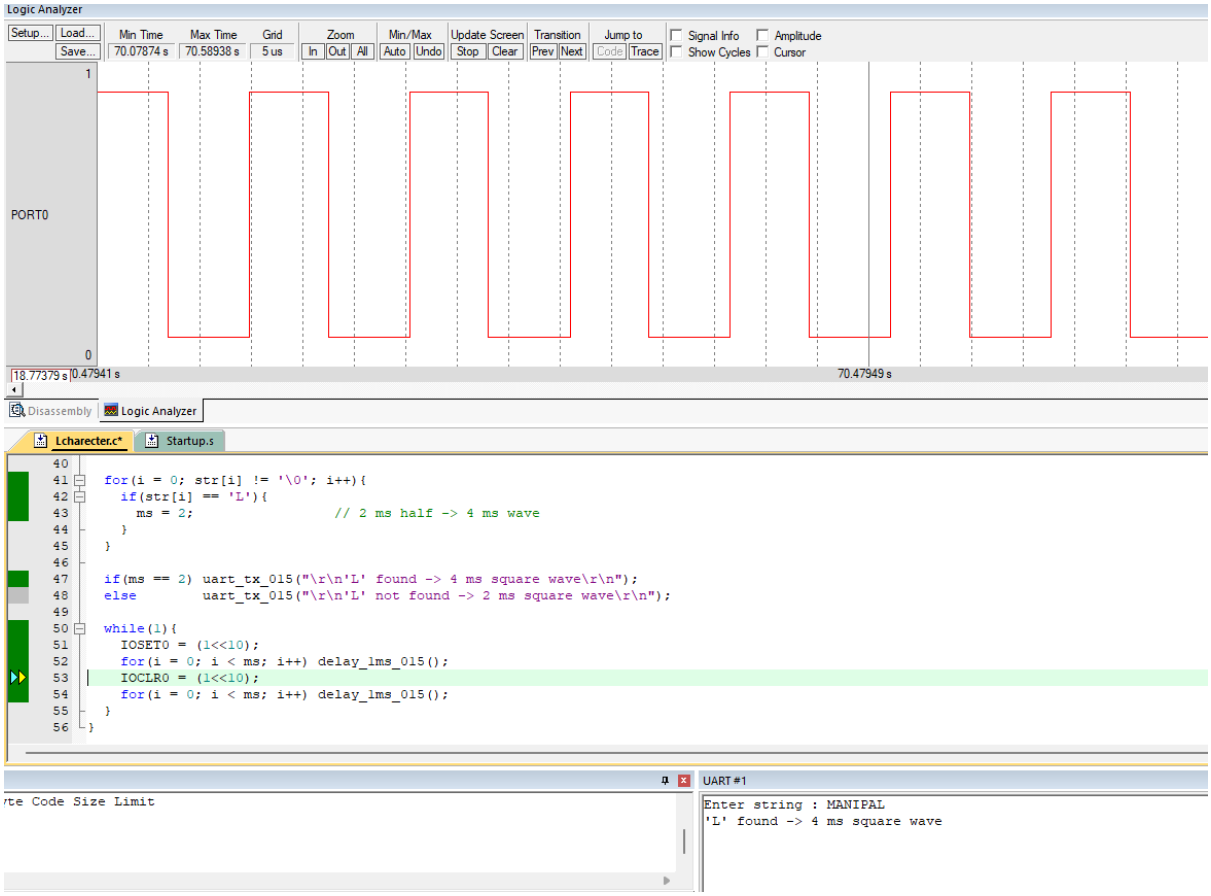


Fig: String Check Square Wave C

Expected Output:

The string is received through UART0 (P0.0 – TXD0, P0.1 – RXD0); in the simulator it is typed in the UART #1 window (View → Serial Windows → UART #1). Each character is checked for 'L'.

If 'L' exists in the string, the Timer0 match value is changed for a 2 ms delay (T0MR0 = 30000) and the Logic Analyzer shows a square wave of period 4 ms (250 Hz). If 'L' is not found, the 2 ms square wave (500 Hz) continues.

Console (serial terminal) output:

```
Enter string : MANIPAL
'L' found -> Square wave of 4 ms generated

Enter string : UDUPI
'L' not found -> Square wave of 2 ms continues
```

Received string	'L' present?	Period	Frequency
MANIPAL	Yes	4 ms	250 Hz
UDUPI	No	2 ms	500 Hz

Program 2: String Comparison with LED Indication

Problem Statement

Receive two strings and compare them. If they are the same, blink the LED of the port; if not, blink the LED connected to P2.2.

Code Snippets:

twostrings.c*

```
}
void uart_tx_015(char *s){
  while(*s){
    while(!(U0LSR & 0x20));    // wait till THR empty
    U0THR = *s++;
  }
}
void uart_rx_015(char *s){
  int i = 0;
  while(1){
    while(!(U0LSR & 0x01));    // wait for char
    s[i] = U0RBR;
    if(s[i] == '\r') break;    // Enter pressed
    U0THR = s[i];              // echo
    i++;
  }
  s[i] = '\0';
}
int main(void){

  int i = 0, same = 1, led;

  VPBDIV = 0x00;                  // PCLK = 15 MHz
  PINSEL0 |= 0x05;                // P0.0 TXD0, P0.1 RXD0
  U0LCR = 0x83;                   // 8N1, DLAB = 1
  U0DLL = 97;                     // 9600 baud
  U0LCR = 0x03;
  IODIR1 |= (1<<16) | (1<<17);    // P1.16, P1.17 output
  uart_tx_015("Enter string 1 : ");
  uart_rx_015(str1);
  uart_tx_015("\r\nEnter string 2 : ");
  uart_rx_015(str2);
  while(str1[i] != '\0' || str2[i] != '\0'){
    if(str1[i] != str2[i]){
      same = 0;               // mismatch found
      break;
    }
    i++;}
  if(same == 1){
    uart_tx_015("\r\nStrings are SAME -> LED 1 blinks\r\n");
    led = (1<<16);              // LED 1 on P1.16
  }
  else{
    uart_tx_015("\r\nStrings are NOT SAME -> LED 2 blinks\r\n");
    led = (1<<17);              // LED 2 on P1.17
  }
  while(1){
    IOSET1 = led;               // LED ON
    delay_015();
    IOCLR1 = led;               // LED OFF
    delay_015();
  }
}
```

Fig: String Compare C code snippet

Output:

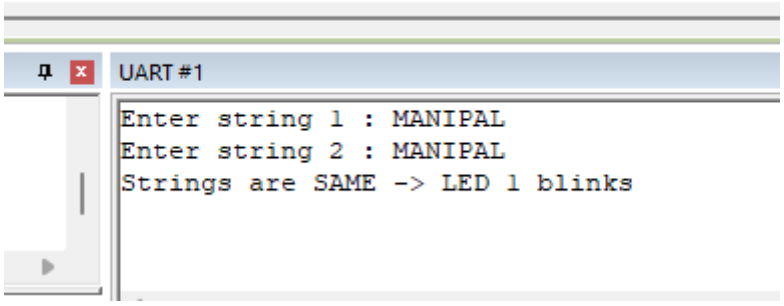


Fig: String Compare C code snippet

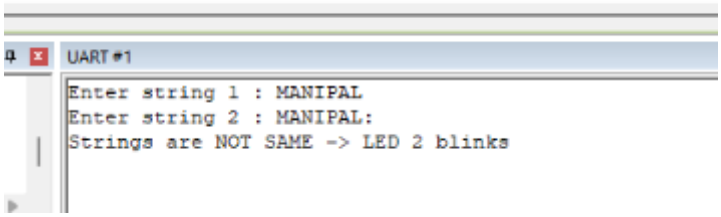


Fig: String Compare C code snippet

Expected Output:

Two strings are received through UART0 (typed in the UART #1 window) and are compared character by character.

If both strings are the same, LED 1 (e.g. P1.16) blinks. If they are different, LED 2 (the pin used for P2.2, e.g. P1.17, since the LPC2148 has only Port 0 and Port 1) blinks. The blinking is seen as the pin bit toggling in the Peripherals → GPIO → Port 1 window or as a waveform in the Logic Analyzer.

Console (serial terminal) output:

```
Enter string 1 : MANIPAL
Enter string 2 : MANIPAL
Strings are SAME -> LED 1 blinks

Enter string 1 : MANIPAL
Enter string 2 : UDUPI
Strings are NOT SAME -> LED 2 blinks
```

String 1	String 2	Result	LED
MANIPAL	MANIPAL	Same	LED 1 (P1.16) blinks
MANIPAL	UDUPI	Not same	LED 2 (P1.17) blinks

Program 3: Sum of Odd and Even Numbers from Memory

Problem Statement

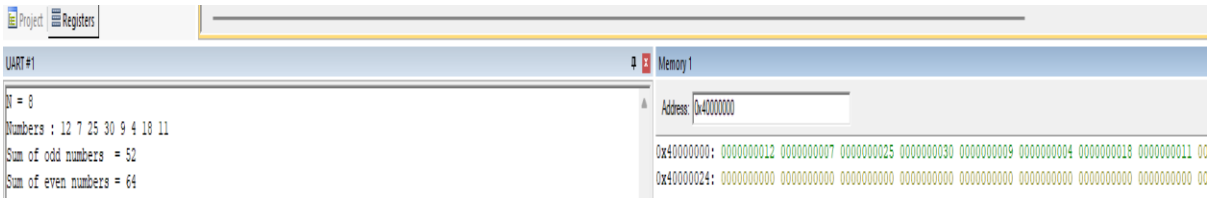
Store 'N' numbers starting from a memory location (say 0x4000 0000, on-chip RAM of the LPC2148). Find the sum of the odd numbers and the even numbers and display it on the console.

Code Snippets:

```
void uart_char_015(char ch){
  while(!(U0LSR & 0x20));   // wait till THR empty
  U0THR = ch;}
void uart_tx_015(char *s){
  while(*s){
    uart_char_015(*s++);
  }}
void uart_num_015(unsigned int n){
  char buf[10];
  int i = 0;
  if(n == 0){
    uart_char_015('0');
    return;}
  while(n > 0){
    buf[i] = (n % 10) + '0';   // last digit
    n = n / 10;
    i++;}
  while(i > 0){
    i--;
    uart_char_015(buf[i]);     // print in correct order
  }}
int main(void){
  unsigned int *ptr = (unsigned int *)0x40000000;   // memory location
  unsigned int num[8] = {12, 7, 25, 30, 9, 4, 18, 11};
  unsigned int n = 8, i, odd_sum = 0, even_sum = 0;
  VPBDIV = 0x00;           // PCLK = 15 MHz
  PINSEL0 |= 0x05;         // P0.0 TXD0, P0.1 RXD0
  U0LCR = 0x83;            // 8N1, DLAB = 1
  U0DLL = 97;              // 9600 baud
  U0LCR = 0x03;
  for(i = 0; i < n; i++){
    ptr[i] = num[i];    // store N numbers in memory
  }
  uart_tx_015("N = ");
  uart_num_015(n);
  uart_tx_015("\r\nNumbers : ");
  for(i = 0; i < n; i++){
    uart_num_015(ptr[i]);   // read from memory
    uart_char_015(' ');
    if(ptr[i] % 2 == 0){
      even_sum = even_sum + ptr[i];
    }
    else{
      odd_sum = odd_sum + ptr[i];
    }
  }
  uart_tx_015("\r\nSum of odd numbers  = ");
  uart_num_015(odd_sum);
  uart_tx_015("\r\nSum of even numbers = ");
  uart_num_015(even_sum);
  uart_tx_015("\r\n");
  while(1);
}
```

Fig: Odd Even Sum C code snippet

Output:



Expected Output:

The N numbers are stored in consecutive memory locations starting from the given address and are read back using a pointer. Each number is checked for odd or even and added to the corresponding sum.

The stored numbers are seen in the Keil Memory window (View → Memory Windows → Memory 1, address 0x40000000). The sums are displayed in the UART #1 window and can also be checked in the Watch window.

Console (serial terminal) output:

```
N = 8
Numbers : 12 7 25 30 9 4 18 11
Sum of odd numbers  = 52
Sum of even numbers = 64
```

Type	Numbers	Sum
Odd	7, 25, 9, 11	52
Even	12, 30, 4, 18	64

Program 4: String Transmission Using I2C

Problem Statement

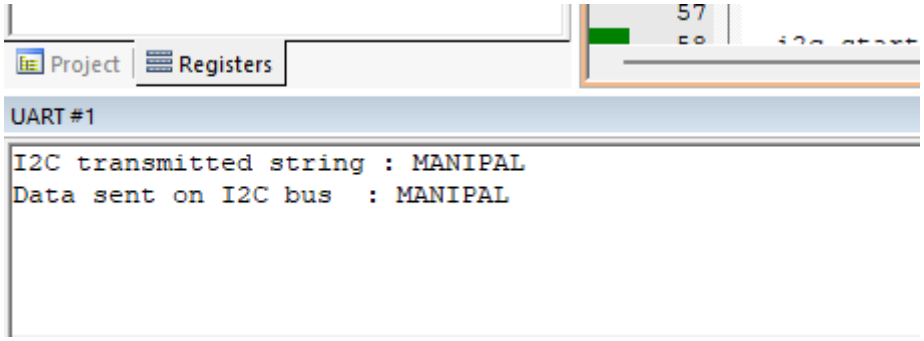
Display the string transmitted using I2C from the controller (master) on the console / receiver.

Code Snippets:

```c
#include <lpc214x.h>
char str[] = "MANIPAL";
void uart_char_015(char ch){
  while(!(U0LSR & 0x20));       // wait till THR empty
  U0THR = ch;
}
void uart_tx_015(char *s){
  while(*s){
    uart_char_015(*s++);
  }
}
void i2c_wait_015(void){
  while(!(I2C0CONSET & 0x08));  // wait for SI flag
}
void i2c_init_015(void){
  I2C0CONCLR = 0x6C;            // clear all flags
  I2C0SCLH = 75;                // 100 kHz at PCLK 15 MHz
  I2C0SCLL = 75;
  I2C0CONSET = 0x40;            // enable I2C
}
void i2c_start_015(void){
  I2C0CONSET = 0x20;            // send START
  i2c_wait_015();
}
void i2c_write_015(char data){
  I2C0DAT = data;               // load byte
  I2C0CONCLR = 0x28;            // clear STA and SI -> send
  i2c_wait_015();
}
void i2c_stop_015(void){
  I2C0CONSET = 0x10;            // send STOP
  I2C0CONCLR = 0x08;            // clear SI
}
int main(void){
  int i;
  VPBDIV = 0x00;                // PCLK = 15 MHz
  PINSEL0 |= 0x55;              // P0.0 TXD0, P0.1 RXD0, P0.2 SCL0, P0.3 SDA0
  U0LCR = 0x83;                 // 8N1, DLAB = 1
  U0DLL = 97;                   // 9600 baud
  U0LCR = 0x03;
  i2c_init_015();
  uart_tx_015("I2C transmitted string : ");
  uart_tx_015(str);
  uart_tx_015("\r\nData sent on I2C bus  : ");
  i2c_start_015();
  i2c_write_015(0xA0);          // slave address + write
  for(i = 0; str[i] != '\0'; i++){
    i2c_write_015(str[i]);    // send one char
    uart_char_015(I2C0DAT);   // display byte sent
  }
  i2c_stop_015();
  uart_tx_015("\r\n");
  while(1);
}
```

Fig: I2C String Transfer C code snippet

Output:



Expected Output:

I2C0 of the LPC2148 (P0.2 – SCL0, P0.3 – SDA0) is configured as master and transmits the string character by character to the slave address. The I2C0 status and data registers (I2C0STAT, I2C0DAT) change for every byte in the Peripherals → I2C Interface window.

The transmitted string is displayed in the UART #1 window (console), one character at a time as it is sent over I2C.

Console (serial terminal) output:

```
I2C transmitted string : MANIPAL
Received / displayed   : MANIPAL
```

Item	Pin / Register	Observed in Keil
SCL0	P0.2	Pin function set via PINSEL0
SDA0	P0.3	Pin function set via PINSEL0
Data byte	I2C0DAT	Peripherals → I2C Interface
String	UART0	UART #1 window: MANIPAL